



COMPLEXIDADE DE ALGORITMOS – BIG O

ESTRUTURA DE DADOS

CST em Desenvolvimento de Software Multiplataforma



PROF. Me. TIAGO A. SILVA



LIVRO DE REFERÊNCIA DA DISCIPLINA

- **BIBLIOGRAFIA BÁSICA:**
 - GRONER, Loiane. **Estrutura de dados e algoritmos com JavaScript**: escreva um código JavaScript complexo e eficaz usando a mais recente ECMAScript. **São Paulo: Novatec Editora, 2019.**
- **NESTA AULA:**
 - **Capítulo 15 – Complexidade de Algoritmos**
 - Notação Assintótica
 - Análise da complexidade de algoritmos



O QUE É COMPLEXIDADE DE ALGORITMOS?

- A complexidade de um algoritmo é uma forma de medir o desempenho de um algoritmo em termos de:
 - Tempo de execução (quantas operações ele faz conforme o tamanho da entrada cresce).
 - Uso de memória (quanto espaço ele consome para funcionar).
- Essa análise é importante para sabermos se o algoritmo continuará eficiente mesmo com entradas grandes.



O QUE É NOTAÇÃO ASSINTÓTICA?

- A notação assintótica descreve o comportamento do algoritmo à medida que o tamanho da entrada (n) se aproxima do infinito. Não nos preocupamos com detalhes pequenos ou constantes, e sim com o crescimento geral.
- A mais comum é a Notação Big O (O grande), mas há outras também.
- A Big O representa o pior caso de desempenho de um algoritmo. Ou seja: quanto tempo (ou espaço) o algoritmo pode levar no pior cenário possível.
- Ela ignora constantes e termos de menor ordem porque, para entradas grandes, eles têm pouco impacto.



EXEMPLOS DE NOTAÇÕES BIG O MAIS COMUNS

NOTAÇÃO	NOME	EXEMPLO TÍPICO
$O(1)$	Constante	Acesso direto a um elemento de um array
$O(\log n)$	Logarítmica	Busca binária
$O(n)$	Linear	Percorrer um array uma vez
$O(n \log n)$	Linearítmica	Quicksort, Mergesort
$O(n^2)$	Quadrática	Bubble sort, seleção dupla
$O(2^n)$	Exponencial	Algoritmos de força bruta (como o da Torre de Hanoi)
$O(n!)$	Fatorial	Permutação completa (ex: problema do caixeiro viajante)

EXEMPLOS DE NOTAÇÕES BIG O MAIS COMUNS

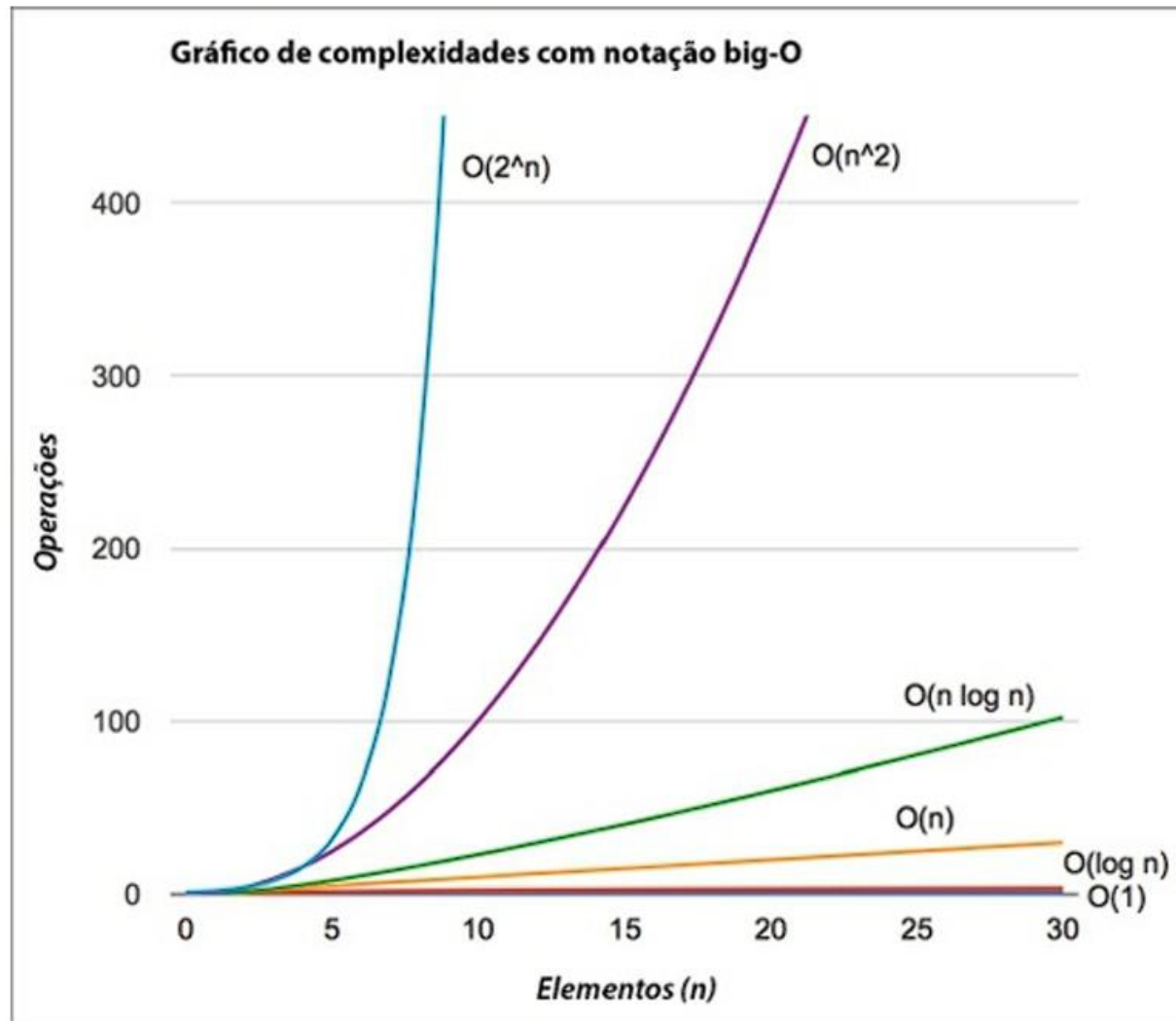


Figura 15.1

EXPLORANDO OS DETALHES DE CADA COMPLEXIDADE

Por que seu código demora para executar?

O QUE É $O(1)$ – COMPLEXIDADE CONSTANTE?

- Um algoritmo ou operação é $O(1)$ quando leva sempre o mesmo tempo para ser executado, independentemente do tamanho da entrada.
- A execução não depende de loops ou recursões baseadas em n (número de elementos).
- Exemplos:
 - Acesso direto a um elemento de um array
 - `const numeros = [10, 20, 30, 40, 50];`
 - `console.log(numeros[3]); // → 40 ( $O(1)$ )`
 - Não importa se o array tem 5, 50 ou 5 milhões de elementos – o acesso é direto pela posição.
 - Inserção em Pilha ou Fila usando **push** ou **unshift**
 - Leitura ou escrita em objetos ou maps (json)
 - Verificação de existência em **Set** ou **Map**

O QUE É COMPLEXIDADE $O(\log N)$?

- $O(\log n)$ significa que o tempo de execução cresce lentamente à medida que a entrada n aumenta.
- Isso ocorre quando o problema é reduzido pela metade (ou outra fração) a cada passo.
- Em vez de processar todos os n elementos, o algoritmo descarta grandes partes da entrada rapidamente.
- Exemplo:
 - A busca binária em um array ordenado é um ótimo exemplo: Mesmo com 1.000.000 de elementos, a busca binária precisa de no máximo cerca de $\log_2(1.000.000) \approx 20$ comparações.
- Como saber se um algoritmo é $O(\log n)$?
 - Se algoritmo divide a entrada pela metade (ou outro fator constante) a cada passo.
 - Se usa estrutura balanceada (como árvores AVL).
- **Atenção:** só vale para entradas ordenadas ou estruturas adequadas

O QUE É COMPLEXIDADE $O(N)$?

- $O(n)$ significa que o tempo de execução cresce linearmente com o tamanho da entrada n . Se você duplicar a entrada, o tempo de execução também dobra (aproximadamente).
- Exemplos:
 - Percorrer um array com **for** ou **forEach**
 - Usar **.includes()** em arrays
 - Somar todos os elementos de um vetor
 - Buscar um elemento manualmente
- O que causa $O(n)$?
 - Loops simples sobre os dados
 - Funções de alta ordem como **.map()**, **.filter()**, **.reduce()**
 - Operações que exigem visitar cada elemento ao menos uma vez

O QUE É COMPLEXIDADE $O(N \log N)$?

- $O(n \log n)$ indica que o algoritmo realiza duas coisas principais: divide a entrada repetidamente em partes menores e processa cada elemento em cada nível da divisão.
- Isso resulta em um tempo total de execução proporcional a $n * \log n$.
- Exemplos Clássicos:
 - Merge Sort
 - Quick Sort
- Como identificar algoritmos $O(n \log n)$?
 - O algoritmo divide a entrada (recursivamente ou por partição)? $\rightarrow \log n$
 - E também processa todos os elementos a cada etapa? $\rightarrow n$
- Quando buscar algoritmos $O(n \log n)$?
 - Precisar ordenar grandes quantidades de dados
 - Está lidando com algoritmos de divisão e conquista
 - Deseja eficiência sem comprometer legibilidade

O QUE É COMPLEXIDADE $O(N^2)$?

- $O(n^2)$ significa que o tempo de execução cresce proporcional ao quadrado do tamanho da entrada. Se você dobrar a entrada n , o tempo de execução quadruplica.
- Se $n = 10$, o algoritmo faz até 100 operações. $O(n^2)$ se torna impraticável com entradas grandes: se $n = 1.000$, ele pode fazer $1.000.000$ operações.
- Quando isso acontece?
 - Sempre que você tem dois loops aninhados (um dentro do outro) percorrendo a mesma entrada:
- Exemplos:
 - Bubble Sort
 - Verificar se há elementos duplicados (sem **Set**)
- Como otimizar?
 - Substituir Bubble Sort por Merge Sort
 - Usar estruturas como **Set** para verificar duplicatas em $O(n)$
- Como identificar um algoritmo $O(n^2)$
 - Há dois loops aninhados percorrendo n ?
 - Há comparação de todos os pares possíveis?
 - Há operações repetidas sem redução de trabalho por iteração?

O QUE É COMPLEXIDADE $O(2^n)$?

- $O(2^n)$ – uma das mais ineficientes e que cresce de forma exponencial com o aumento da entrada. Ela aparece em algoritmos que exploram todas as combinações possíveis de um conjunto.
- $O(2^n)$ indica que, à medida que o tamanho da entrada n cresce, o tempo de execução dobra a cada novo elemento.
 - Se $n = 5$, o algoritmo faz até $2^5 = 32$ operações
 - Se $n = 10$, ele faz $2^{10} = 1024$
 - Se $n = 30$, ele faz $2^{30} \approx 1$ bilhão de operações
- É comum em algoritmos de força bruta, como:
 - Geração de subconjuntos
 - Problemas de decisão que testam todas as possibilidades
 - Soluções recursivas sem otimização
- Dica para reconhecer $O(2^n)$:
 - Está usando recursão sem controle
 - Explora todas as combinações possíveis
 - Cada chamada recursiva gera duas (ou mais) chamadas subsequentes

O QUE É COMPLEXIDADE $O(N!)$?

- A complexidade $O(n!)$ (fatorial de n) é uma das mais lentas e impraticáveis para entradas acima de 10 a 12 elementos. Ela ocorre em algoritmos que precisam gerar todas as permutações possíveis de uma entrada.
- $O(n!)$ indica que o número de operações cresce como o fatorial de n , ou seja:

$$n! = n \times (n - 1) \times (n - 2) \times \dots \times 1$$

- Por exemplo:
 - $3! = 3 \times 2 \times 1 = 6$
 - $5! = 120$
 - $10! = 3.628.800$
 - $20! \approx 2.4 \times 10^{18}$
- Você provavelmente está lidando com $O(n!)$ se:
 - Está gerando todas as permutações ou arranjos de n elementos
 - Está tentando resolver o Caixeiro Viajante com força bruta
 - Está testando todas as ordens possíveis de um conjunto

ANÁLISE DA COMPLEXIDADE DE CADA ESTRUTURA

Qual escolher não depende apenas da complexidade em implementar

POR QUE ISSO IMPORTA?

- Imagine dois algoritmos que funcionam bem para 100 elementos. Mas se a entrada crescer para 1 milhão:
 - Um algoritmo $O(n)$ ainda pode funcionar.
 - Um algoritmo $O(n^2)$ vai demorar absurdamente mais.
- Logo, analisar a complexidade ajuda a escolher soluções mais escaláveis.



COMPLEXIDADE EM ESTRUTURAS DE DADOS COMUNS

ESTRUTURA	ACESSO	INSERÇÃO	REMOÇÃO	BUSCA	OBSERVAÇÕES
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	Inserção/remoção no meio exige deslocamento
Pilha	$O(n)$	$O(1)$	$O(1)$	$O(n)$	LIFO (Last-In, First-Out)
Fila	$O(n)$	$O(1)$	$O(1)$	$O(n)$	FIFO (First-In, First-Out)
Set	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Elementos únicos, sem ordem garantida
Map (Objeto/Map)	$O(1)$	$O(1)$	$O(1)$	$O(1)$	Chave-valor
Lista Encadeada (simples)	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$	*Se tiver ponteiro para o nó alvo
Lista Duplamente Encadeada	$O(n)$	$O(1)^*$	$O(1)^*$	$O(n)$	Ponteiros para anterior e próximo; facilita remoção
Árvore Binária de Busca (BST)	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	$O(\log n)^*$	*Se balanceada; pior caso $O(n)$
Árvore AVL	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(\log n)$	Auto-balanceadas
Grafos (matriz adjacência)	$O(1)$	$O(1)$	$O(1)$	$O(n^2)$	Melhor para grafos densos, alto uso de memória

OUTRAS NOTAÇÕES ASSINTÓTICAS

- Além da Big O (pior caso), temos:
 - $\Omega(n)$ – Ômega: o melhor caso.
 - $\Theta(n)$ – Teta: o caso médio ou limite superior e inferior ao mesmo tempo.
- Na prática, usamos $O(n)$ quase sempre porque queremos saber qual o pior cenário possível.



OBRIGADO!

- Encontre este **material on-line** em:
 - Slides: Plataforma Microsoft Teams
- Em caso de **dúvidas**, entre em contato:
 - **Prof. Tiago:** tiago.silva238@fatec.sp.gov.br

